IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 17 April 2025, accepted 8 May 2025, date of publication 15 May 2025, date of current version 23 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3570359

RESEARCH ARTICLE

Automated Classification and Identification of Non-Functional Requirements in Agile-Based Requirements Using Pre-Trained Language Models

ABDULRAHIM ALHAIZAEY[1,2] AND MAJED AL-MASHARI[1], (Member, IEEE)
[1]Department of Information Systems, College of Computer and Information Sciences, King Saud University, Riyadh 11451, Saudi Arabia
[2]Department of Information Systems, King Khalid University, Abha 61421, Saudi Arabia

Corresponding author: Abdulrahim Alhaizaey (442105999@student.ksu.edu.sa)

This work was supported in part by King Saud University (KSU); and in part by King Khalid University (KKU), Saudi Arabia. The work of Abdulrahim Alhaizaey was supported by KKU through the Ph.D. Scholarship at KSU.

ABSTRACT Non-functional requirements (NFRs) are critical factors for software quality and success. A frequently reported challenge in agile requirements engineering is that NFRs are often neglected due to the focus on functional requirements (FRs) and the limited capability of agile requirements documented as user stories to represent NFRs. With the emergence of transfer learning and large pre-trained language models, various applications in requirements engineering have become feasible, alleviating several longstanding challenges. This study evaluates transformer-based models for the automated identification and classification of NFRs. We leveraged transfer learning with pre-trained transformer models to automate the identification and classification of NFRs in agile textual requirements documented as user stories. A dataset of over 10k user stories was collected and labeled, and pre-trained transformer models, including BERT, RoBERTa, XLNet, and DistilBERT, were fine-tuned to automate the identification of NFRs. We incorporated Focal Loss during training to mitigate the dominance of functionally driven requirements and class imbalances. In addition, thorough experiments on hyperparameter optimization were employed using Bayesian hyperparameter optimization to obtain the combination of hyperparameters that best correlated with the aim of enhancing each model's performance. Our evaluation demonstrated that the finetuned pre-trained models significantly outperformed comparable prior approaches relying on rule-based techniques or traditional machine learning, with a fine-tuned BERT model achieving an F1 Score of 93.4 %. These findings highlight the potential of pre-trained language models in agile requirements engineering, enabling more efficient NFRs identification, reducing manual review burden, and facilitating a viable and efficient approach to address the neglect of NFRs in agile development processes.

INDEX TERMS Nonfunctional requirements, agile software development, requirements engineering, user stories, transfer learning, transformers, pre-trained language models, BERT, XLNet.

I. INTRODUCTION

Agile software development (ASD) approaches have gained wide popularity in the software industry. These approaches are even considered today as the predominant development methodologies among software development practitioners [1], [2], [3]. While there is undeniable evidence that ASD alleviates some requirements engineering (RE) challenges faced in traditional plan-driven development, such as requirements changes and specifying requirements in advance, new challenges have been introduced and emerged [4]. One of the most frequently reported challenges in agile RE literature is

The associate editor coordinating the review of this manuscript and approving it for publication was Mahmoud Elish.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

the neglect of non-functional requirements (NFRs) in agile RE. Studies [5], [6] have revealed that neglecting NFRs is a significant challenge in agile RE. Schön et al. [7] reported that 47.4% of agile industry experts acknowledged that "establishing NFRs is challenging."

NFRs are essential success factors in software projects. The current practice of realizing them in agile RE relies mainly on teams' tacit knowledge [8]. Studying this issue, Alsaqaf et al. reported that existing proposals are often too heavy or lack agility assessments, which may reduce their applicability and adoption among agile practitioners [8]. Neglecting NFRs in agile RE has been attributed to several factors. For example, user stories, the most widely used technique for specifying requirements in ASD, [9], [10], [11], [12] have been criticized for their inability to represent NFRs sufficiently. Their primary focus on functional requirements (FRs) leads to the omission or inadequate handling of NFRs [13]. Consequently, NFRs are often managed tacitly or poorly documented. The study [8] reported that ASD has no widely accepted technique for specifying NFRs; it usually relies on the product owner or team members to identify NFRs, which may be problematic if the team lacks adequate NFRs knowledge. Nonfunctional needs, such as security, performance, or scalability, could present challenges that influence architectural choices, raising concerns about earlier hardware and software selection. Therefore, developers must consider the potential cost and recognize this feature early to avoid future complexity. The system's adherence to such critical NFRs necessitates early identification so that they can be communicated and addressed appropriately.

Given that manual review and classification of requirements are time-consuming and susceptible to error, automated classification supported by advanced artificial intelligence techniques could help provide a solution to this RE problem. A brief review of the scientific literature shows that NLP and AI techniques has recently begun to establish a position in RE research.

Software and system requirements are traditionally specified and documented using natural languages [14]. In agile environments, user stories represent requirements as a textual description of functionality that is supposed to be achieved by the software under development. The popularity of user stories among agile practitioners, their simplicity, and their concise structure make them ideal candidates for promising NLP analysis and research [15]. Dalpiaz et al. [16] indicated that with the current availability of NLP tools and the growing amount of available data, there are unprecedented opportunities to use advanced AI techniques in RE practice to help automatically analyze requirement documents and solve real-world RE problems. A broad range of applications in RE can be accomplished using NLP, such as requirements classification, identifying ambiguous requirements, and requirements prioritization, to name a few.

This study proposes an approach for identifying NFRs in user stories, the most common form of requirements documentation in ASD. Our approach leverages pre-trained transformer-based models to automate the prediction and classification of NFRs in agile-specified requirements, aiming to address the main challenges identified in the literature: limited ASD and NFRs documentation, insufficient representation of NFRs in user stories, the functional focus of user stories, and a lack of NFR knowledge among agile practitioners and stakeholders.

To address the problem of neglecting NFRs in agile textual requirements, we propose an automation-based approach using pre-trained language models to identify and classify NFRs in agile textual requirements based on an annotated dataset for NFRs in an agile-based format. We demonstrate implementing, fine-tuning, and evaluating the pre-trained language models to identify NFRs in user stories. First, we explored different agile requirements resources, collected them, and labeled them. We fine-tuned and evaluated four pre-trained transformer-based models (BERT, RoBERTa, XLNet, and DistilBERT) as part of our domain-specific classification approach. Each model was adapted to our annotated dataset to assess its effectiveness in identifying NFRs in user stories. Finally, we evaluated the performance of these models and compared them with state-of-the-art results in the literature. The primary contributions of this study are as follows:

- We introduced a labeled dataset for NFRs in the format of agile specifications. Inspired by the agile requirements datasets [17], we undertook an extensive search, collection, aggregation, labeling, and release of user stories dataset labeled with NFRs. Our dataset contains more than 10K samples and has been used for training the models investigated in this research to address the challenge of identifying NFRs in agile RE. We have made it available in this project repository, and it could help other researchers working on the agile specification, not only in identifying NFRs but in other agile requirements research as well. The detailed and thorough process of constructing and labeling this dataset is explained in this paper, instilling confidence in its quality and reliability
- We extensively designed, finetuned, and trained different pre-trained language models to evaluate and demonstrate their applicability in identifying NFRs within user stories. This includes extensive work on hyperparameter optimization and exploration of different techniques to handle dataset imbalance.
- We provided a thorough evaluation of the applied fine-tuned models, demonstrating their effectiveness and potential. Our benchmarking with relevant literature showed that they outperformed existing state-of-the-art approaches in identifying NFRs within agile requirements. This contribution establishes the viability of using pre-trained language models to enhance the identification of NFRs in agile contexts.

The remainder of this paper is organized as follows. Section II provides relevant background on agile RE, NFRs, and AI

for RE. Section III provides a literature review relevant to this study. Our research methodology, which includes dataset construction and labeling, dataset preprocessing, input formatting, and models training, is outlined in Section IV. Section V reports our results, findings, and comparative analysis with current state-of-the-art methods. Section VI discusses our findings' implications and outlines future work directions. The concluding remarks and future work are presented in Section VIII.

II. BACKGROUND

A. AGILE RE

Agile RE has challenged traditional RE practices in response to ASD manifesto [18]. There is no common definition of agile RE, and most agile approaches do not explicitly discuss RE but cover RE practices in their general method descriptions [4]. Inayat et al. [5] indicated that the expression "agile requirements engineering" is adopted to describe an "agile way" of planning, executing, and reasoning RE activities. However, the core traditional RE phases of elicitation, analysis, verification and validation are present in all agile processes but with variant techniques [19]. Agile RE differs from traditional RE because it takes an iterative discovery approach [20]. Cao and Ramesh conducted an empirical study [20] to study and analyze Agile RE practices in 16 software development organizations. They reported seven Agile RE practices as the most adopted practices in Agile RE. The adopted practices are summarized as follows:

- Face-to-face communication, instead of written specifications, utilizes simple techniques like user stories to define high-level requirements.
- Iterative RE, where requirements emerge during development rather than being predefined.
- Extreme prioritization based on the customers' defined business value requirements.
- Managing requirements change via constant planning and feedback from customers.
- Using prototypes to communicate, validate, and refine requirements with customers.
- Test-driven development.
- Frequent review meetings for requirements validation.

Software and system requirements are typically specified and documented using natural language [14]. In agile environments, user stories are the most widely used technique for specifying requirements [9], [10], [11], [12]. A user story is a short, written textual description of functionality that is supposed to be achieved by the software under development. User stories are a powerful tool for communicating the requirements between product owners and development teams. Lucassen et al. [21] found that using user stories improves practitioners' work productivity. They also found that most practitioners who adopt user stories follow the original user-story template suggested in Cohn's work [22]. This template captures who the user of the functionality is, what it expects from the system, and, optionally, why it is important [23]. The template of the User Story is formulated as follows: "As a ⟨user ⟩, I want ⟨goal⟩, [so that ⟨reason⟩]". A user story is conceptually modeled, as depicted in Figure 1 [24], where the agile textual requirement is formulated with a role, means, and end. The role part represents a persona or stakeholder who asks for the specific feature of the system under development. Means have the following common elements: a subject with an aim, an action verb expressing a requested feature and a direct object on which the subject executes the action. The end part, which starts with "…, so that …", explains the benefit or why the means are requested.

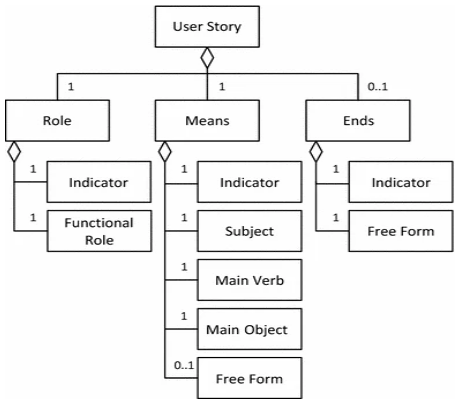

FIGURE 1. Conceptual anatomy of a user story [24].

The popularity of user stories among agile practitioners, their simplicity, and concise structure makes them ideal candidates for promising NLP analyses and research [15]. While agile requirements are usually specified using user stories, user stories are widely criticized for their inability to represent NFRs because of their functionality-driven nature. This study explored this issue wisely and pragmatically.

B. NON-FUNCTIONAL REQUIREMENTS (NFRs)

A widely acknowledged categorization among RE researchers and practitioners is that software requirements are classified into Functional Requirements (FRs) and Non-Functional Requirements (NFRs). FRs represent statements of the services to be provided, whereas NFRs represent constraints on or quality attributes of these services [25]. Examples of NFRs include usability, availability, integrity, security, accuracy, performance, efficiency, and user-friendliness. Although there is no consensus on the definition of NFRs in the RE community, there is consensus that NFRs are critical for the success of a project [26].

Notably, functionality is generally not useful or usable without the required NFRs [27]. In other words, NFRs cannot be independent and must always refer to one or more FRs. It is widely noted in the literature that NFRs and quality requirements are used interchangeably. Given this and for this research, we preferred to use the term NFRs to avoid such ambiguity. The body of knowledge on NFRs is widely scattered in the software engineering literature, software quality assurance, and software quality models.

C. AI FOR AGILE RE

Given that agile requirements are specified using natural language in the format of user stories, language models are a candidate tool for application here. A systematic literature review [28] reviewed the recent state-of-the-art NLP research on user stories. Their results revealed that the use of NLP in user stories can be categorized into four different directions and use cases: discovering defects and deviations, generating software artifacts or models from them, tracing links or dependencies between user stories and other software artifacts, and identifying key abstractions from user stories.

With the emergence of Large Language Models, Hou et al. [29] surveyed the applications of LLMs in various Software Engineering tasks. They classified the LLM research on RE into ambiguity treatment, term identification, requirements classification, co-reference detection, requirements analysis, requirements evaluation, and traceability automation.

Dalpiaz et al. [16] indicated that with the current availability of NLP tools and the growing amount of available data, there are unprecedented opportunities to use advanced AI techniques in RE practice to help automatically analyze requirements documents and solve real-world requirements engineering problems. A brief glance at the relevant scientific literature shows that the leverage of AI techniques -generally via a combination of NLP and machine learning approaches to RE problems–has started to find its way into RE research and practice. Overall, advances in AI for working on NLP have promising potential for improving different RE tasks. Given that manual review and classification of requirements is time-consuming, automated classification supported by advanced AI techniques could support the development of active and mature solutions to tackle RE problems.

III. LITERATURE REVIEW

Early studies have been conducted on automating software requirement classification. One of the first studies [30] introduced an NFR classifier to classify NFRs using indicator terms. Their method was based on learning a set of terms for each type of NFR. The indicator terms were extracted, and their probability weights were calculated based on their frequency within a specific requirement text. The method involves two main stages: extracting the indicator terms and then using a classifier to classify the requirements based on the probability weight. The classifier assigned a class according to the highest probability weight. The type is considered functional FR if it does not exceed a classification threshold. Evaluation of their classifier showed acceptable results, which paved the way for subsequent studies to improve the classification of NFRs in software specifications. Similarly, the NFR locator approach [31] uses requirement specifications, requests for proposals (RFP), or user manuals as inputs. Sentences are then classified into specific NFR categories using a support vector machine algorithm. Their support vector machine algorithm is twice as effective as a multinomial naïve Bayes classifier.

Handa et al. [32] proposed a framework for predicting and classifying NFRs using M-LSTM and CNN. Similarly, a deep neural network was proposed [33] to classify NFRs from software requirement documents. Both studies aimed to facilitate and improve the classification of NFRs in traditional development requirements, such as a request for proposal documents (RFP) and software requirements specification (SRS), but without specific consideration for requirements documented in user stories format. Other studies have also suggested management practices. Silva et al. [34] proposed a process for handling the elicitation of NFRs. Their process guide involves defining the application context, identifying experts to consult, defining NFR characteristics, establishing default NFRs, validating with users, and making the guide available for organizational use.

In the agile development landscape, research proposals vary widely, from management approaches to specific facilitating artifacts to dedicated guidelines for mitigating the problem of NFR neglect. Kopczynska and Nawrocki [35], for example, suggested supporting agile teams with specific NFRs templates to identify NFRs. The NORMAP methodology [36] proposed a conceptual framework for modeling NFRs in agile processes using the W8 User Story Card Model and Agile Requirement taxonomy, which classifies requirements as either functional (Agile Use Case) or non-functional (Agile Loose Case) and agile choice case to represent the potential of NFRs. The NERV method, [37] abbreviated from "Non-functional Requirements Elicitation, Reasoning, and Validation in Agile Processes," suggested a companion NFR card, a trigger NFR card, and a reasoning taxonomy to manage NFRs in ASD. Farid and Mitropoulos [38] suggested a risk-driven planning scheme to be used as a prioritization approach for planning NFRs early in ASD. Behutiye et al. [39] presented a guideline containing 12 recommendations for documenting quality requirements for ASD. Three recommendations focus on raising awareness of the significance of quality requirements among practitioners and stakeholders. The other nine recommendations suggest facilitating artifacts and practices to highlight important aspects of documenting such requirements.

Younas et al. [40] introduced a management guideline for specifying NFRs similar to the process for handling the elicitation of NFRs [34] but adjusted it to agile methods. A similar work [41] presented a guideline of eight steps, including identifying NFRs using a glossary and expert involvement. A framework for handling NFRs in ASD was proposed [42]. This framework consists of a tool that uses historical development data to recommend candidate NFRs. The recommended NFRs were then discussed with stakeholders in an NFR session before being validated and documented in a specific NFR user story. It has been argued [8] that introducing many new artifacts to handle NFRs in agile environments does not keep the process agile and results in a heavily documented non-agile approach. The proposed approaches for agile methods either introduce many new artifacts or lack agility assessment.

There have been recent efforts to automate the identification of NFRs in agile-formulated requirements as user stories. Ahmed et al. [43], [44] proposed an NLP-based framework to automatically extract and prioritize software quality attributes for ASD. Their framework, QAExtractor, uses NLP and regular NFR expression repositories to parse user stories and indicate quality attributes before prioritizing them based on their frequency, stakeholder impact, and criticality. The framework was validated through two case studies with a precision of 0.76, recall of 0.92, and F1-score of 0.86. The results highlight the effectiveness of the framework in automating quality attribute extraction in ASD, reducing reliance on experts, and improving early software quality assessment.

Gilson et al. [45] showed that user stories contain information about quality attributes. Therefore, they explored ways to identify stories that include such information automatically. They used a Convolutional Neural Network to train on a small dataset containing 40% samples labeled with Compatibility, Security, Usability, Maintainability, Performance, Portability, and Reliability, and the remaining 60% as FR. Their evaluation showed 0.74 precision, 0.41 recall, and 0.53 F1-score. They advocated machine learning to automate and accelerate the classification of user stories with quality attributes to avoid finding architectural problems late in development projects and to support the process of making architectural design decisions. Similarly, a machine learning model based on spaCy NLP was used in [46] to identify quality attributes in 1675 labeled stories automatically. Their evaluation showed an average precision of 0.65, average recall of 0.67, and average f1 score of 0.66. Their study indicated that the automatic identification of quality attributes could reduce the effort of initial backlog analysis and provide valuable information about quality attributes for better architectural decision-making.

Other research efforts have been dedicated to studying specific NFRs in agile RE. For instance, Villamizar et al. [47] designed an approach for reviewing security-related aspects in the agile requirement specifications of web applications. First, their proposal used NLP to parse user stories and extract specific security terms. Then, it relates the user stories with identified security properties to the security requirements obtained from the Open Web Application Security Project (OWASP) using a rule-based NLP technique to be reviewed for detecting defects. Casillo et al. [48] presented an approach for automatically detecting privacy requirements in user stories. They used NLP tools to extract private information in user stories, a pre-trained used a convolutional neural network to process this information and utilized the transfer learning capabilities of pretrained models for privacy disclosure detection in user stories. Herwanto et al. [49] used BERT and RoBERTa language models to identify the disclosure of privacy requirements in the user stories.

Despite these valuable efforts and proposals, identifying NFRs in agile environments still needs to be more agile-friendly and consistent with agile practices. The emergence of large-language pre-trained models represents a breakthrough in RE research and practice. This breakthrough could contribute to solving many challenges associated with requirements engineering, including the problem of neglecting NFRs in agile requirements documentation, which is what this paper demonstrates and presents. The aforementioned research gaps have motivated the development and evaluation of an approach for reviewing and improving NFRs in user stories. This research proposal has taken a step forward in addressing this gap. Unlike prior studies that relied on rule-based systems or traditional ML, our approach fine-tunes transformer models for the contextual understanding of NFRs in user stories.

IV. RESEARCH METHODOLOGY

This section presents the methodology used in this study, as illustrated in Figure 2. First, we constructed, studied, and annotated the datasets needed for this research. We then processed the dataset texts before preparing them for tokenization and embedding for each pre-trained model. Subsequently, several pre-trained language models were trained and fine-tuned, and the hyperparameters were optimized to leverage the best capabilities of the models. Then, the evaluation data were collected for analysis and benchmarked with the relevant literature. The following sections explain each stage in detail, outlining the headlines of the research methodology depicted in Figure 2.

A. DATASET COLLECTION

Although research involving AI in the domain of agile RE shows promise, one of the primary challenges is the lack of a sufficient dataset of agile user story requirements.

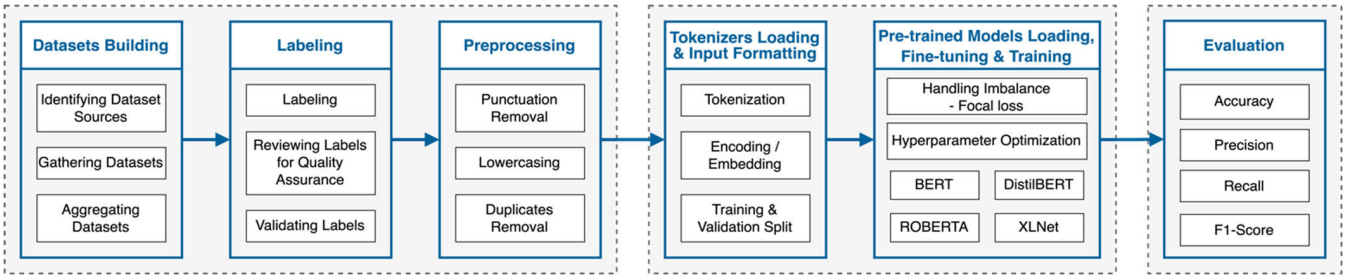

FIGURE 2. Research methodology and stages.

TABLE 1. Dataset sources.

Data Source	Number of samples
Dalpiaz [17]	1680
Murukannaiah et. al [50]	3267
Aydimer [51]	773
Rohmann and Paech [52]	32
Lucassen et al. [24]	465
Lucassen et al. [23]	187
Prabu et al. [53]	602
Dalpiaz et al. [54]	540
The Trident Project[1]	49
Online Contents[2]	733
An ongoing project[3]	1779
Duplicates	-101
Total	10006 sample

The currently available user story datasets are either limited, private, or small in size [28]. Therefore, we needed to construct a dataset that would allow us to conduct our study. To build this dataset, we explored academic resources and various online sources that provide agile requirements in user story formats. We then collected them together in preparation for the study and for subsequent labeling. Table 1 presents the sources of the collected dataset along with the total amount of data acquired. Our dataset, implementation, and results are available on our GitHub[4] repository.

B. DATASET LABELING

The labeling process requires considerable effort and analysis because the context of some backlogs and their domains is unclear. For this purpose, we divided the labeling strategy into different stages, as illustrated in Figure 3. In the first stage, we analyzed the requirement datasets collected from previously identified resources. We uploaded each backlog to a spreadsheet file and performed a brief reading and analysis of the backlog requirements. Each backlog was studied separately to identify the backlog domain. This analysis helps in realizing the system domain, the scope of its requirements, and whether user stories can indicate or imply NFRs. Having realized that some NFRs could be represented in the statements, we started identifying NFRs categories. Because NFRs are widely scattered in the SE literature, software quality assurance, and software quality models, we used the quality ISO/IEC 25010 [55] model as a reference for our labeling process. This model provides a comprehensive, detailed, and organized reference for NFRs. It comprises nine quality characteristics (NFRs), which are decomposed into 32 subcategories, as shown in Table 2. Table 3 presents the descriptions of the NFRs described in the model document.

In the second stage, an actual NFRs labeling process was performed. Two researchers performed the labeling process and an expert in the field reviewed the labels. The labeling process starts by reading each textual requirement and asking: "Does the requirement statement explicitly or implicitly necessitate, indicate, or imply any NFR?". If true, we labeled it as NFR, while the statement is labeled as FR if the answer is no. After separating the FR and NFR labels, the next round was the specific NFR labeling. To facilitate this, we codified the FR and NFRs in spreadsheets. We then questioned each NFR-labeled statement: "What is the specific NFR that is explicitly or implicitly needed or implied to this statement?" We label this with one of the labels in Table 2.

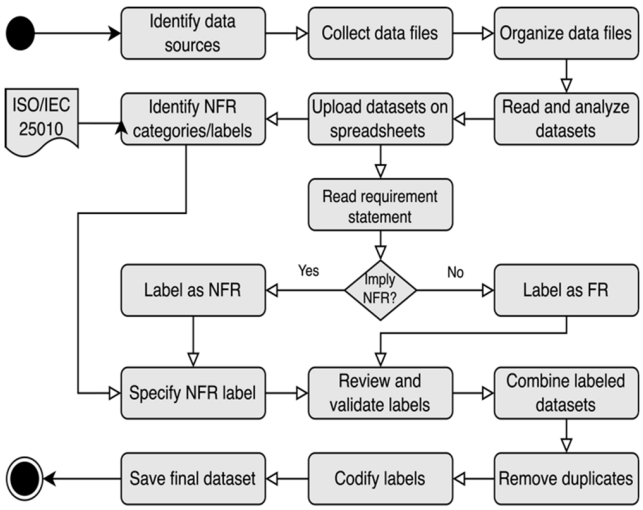

FIGURE 3. Dataset construction and labeling process.

The Functional Suitability characteristic defined in ISO/IEC 25010 was not included in our dataset's labeled NFR categories. While it is considered a quality attribute in our labeling reference, our study focused on NFRs that are more readily identifiable through textual reading and linguistic analysis. Additionally, since the backlogs used in this research were written by others and collected from various public datasets, it is difficult to assess the functional suitability with confidence in its related dimensions (correctness, completeness, and appropriateness). "FR" in our dataset refers specifically to general functional requirements rather than the Functional Suitability dimension.

To assess the reliability of the labeling process, we retrospectively computed the agreement between the initial consensus labels (assigned collaboratively by two researchers) and the expert-reviewed labels using Cohen's Kappa coefficient [56]. The resulting score was 0.8335, which indicates substantial agreement between the two sets of annotations based on established interpretation thresholds. This level of agreement supports the overall consistency and trustworthiness of the labeling procedure. If a label was found to be inaccurate or misclassified by the expert, it was updated accordingly. Although the initial labels were not assigned independently, the subsequent expert review served as a validation step, and the kappa statistic demonstrates that the expert's adjustments were limited in scope. In the final stage, we combined all datasets into a unified file and removed

[1] http://blogs.library.duke.edu/digital-collections/2009/02/13/ on-the-trident-project-part-1architecture/.

[2] Collected from various blogs and material posted online.

[3] An ongoing project we are conducting on automating the generation of acceptance criteria for user stories.

[4] https://github.com/a-re22/Agile_Req_NFRs.

duplicates and numerically codified labels. The complete process of the dataset construction, labeling, and preprocessing is shown in Figure 3.

TABLE 2. Included labels from Iso/Iec 25010 [55].

ID	Abb.	Label	Related Attributes
0	FR	Functional	• Functionality feature
1	CM	Compatibility	• Co-existence • Interoperability
2	MN	Maintainability	• Modularity • Reusability • Analyzability • Modifiability • Testability
3	SE	Security	• Confidentiality • Integrity • Accountability • Authenticity • Non-repudiation
4	PE	Performance	• Time behavior • Capacity • Resource utilization
5	PO	Portability	• Adaptability • Installability • Replaceability
6	RL	Reliability	• Availability • Recoverability • Fault tolerance • Maturity
7	US	Useability	• Operability • Learnability • User interface aesthetics • Appropriateness recognizability • User error protection • Accessibility

TABLE 3. Types of software Nfrs And their definitions [55].

NFR	Description
Functional suitability	Degree to which a product or system provides functions that meet stated and implied needs when used under specified conditions.
Compatibility	Degree to which a product, system, or component can exchange information with other products, systems, or components and/or perform its required functions while sharing the same hardware or software environment.
Maintainability	Degree of effectiveness and efficiency with which a product or system can be modified by the intended maintainers.
Performance efficiency	Degree of performance relative to the amount of resources used under stated conditions
Portability	Degree of effectiveness and efficiency with which a system, product or component can be transferred from one hardware, software or other operational or usage environment to another.
Reliability	Degree to which a system, product or component performs specified functions under specified conditions for a specified period of time.
Security	Degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization.
Usability	Degree to which a product or system can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use.

• ***Note:*** Functional Suitability, while included in ISO/IEC 25010, was excluded from our annotation process due to its evaluation being based on correctness-related aspects of requirements, and it is difficult to assess the suitability of the described functionality reliably.

C. DATASET PREPROCESSING

During the preprocessing and cleaning of datasets, punctuations were removed from the text to minimize unnecessary noise in the datasets and to let models concentrate on meaningful words and patterns. Text entries were converted into lowercase entries to reduce redundancy. Duplicate samples were identified and removed to minimize their impact on model training and to reduce potential bias. A total of 101 duplicate user stories were detected, primarily due to overlapping content across some backlogs shared between the dataset published in [17], [23], and [24]. Removing these duplicates ensured that the final dataset remained representative and free from redundancy. After this step, the resulting dataset comprised 10,006 unique user stories, which were used for training and evaluation. Then, the labels were converted to numerical IDs to simplify text analysis and allow efficient data handling and compatibility with machine learning procedures. Finally, all data files were combined into a unified file, and consistency across records was reviewed to ensure a clean and structured dataset ready for training and analysis.

1) STATISTICS OF THE FINAL ANNOTATED DATASET

To summarize the final dataset we used to train the studied models, FR accounted for approximately 64.5% of the dataset, while NFRs comprised the remaining 35.5%. The dataset averages 25 words per sentence, with the shortest sentence containing seven words and the longest reaching 154 words. Figure 4 shows the distribution of the dataset categories. Table 4 shows examples of agile requirements from the datasets.

We showed two requirements for each NFR to provide an idea of agile requirements and what the labels look like. We labeled the requirements using general NFR categories and did not go down to the level of specific related subcategories because of the limited data samples for subcategories and time constraints. This decision was made after realizing that it was sufficient to use a broad category rather than a specific one from the labeling reference. For example, if a requirement text is labeled with the confidentiality subcategory of security, we use the security label, as it includes confidentiality by default. To ensure proper evaluation and prevent data leakage, the labeled dataset was randomly split into two disjoint subsets: 80% for training and 20% for evaluation.

D. MODELS SELECTION

Transformer pre-trained language models have significantly revolutionized the NLP field. Pre-trained models can be fine-tuned for a variety of downstream tasks while maintaining their general architecture, unlike traditional machine learning models that often require training from scratch. Many pre-trained models exist, each with different strengths and training objectives. As proof of concept for their effectiveness in our specific classification task, we used four

TABLE 4. Agile requirements examples.

Label	Example from the Dataset
FR	1) As a manager, I want to keep track of available products and get notified when they are running low, so I can manage inventory properly. 2) As an archivist, I want to view collection files, so that I can gain background information for processing a collection.
CM	1) As a developer, I want a Mongo integration so that I can integrate packaged data with pipelines that use Mongo. 2) As a researcher working with data, I want a Microsoft Power-BI integration, so that I can import datasets without downloading them locally.
MN	1) As a user, I want the app to be automatically updated so that I always have the latest features and fixes. 2) As a product manager, I want the software tool to be flexible for adding new features, so that it can evolve with market trends.
SE	1) As a data contributor, I want to set access levels for my datasets, so that I can control who views or uses them. 2) As a system administrator, I want users to log in using twostep authentication, so that authorized users are only allowed.
PE	1) As IFA, we want the system response time for fans team administration and IFA administration to not exceed 1 second so that to enhance the user's experience. 2) As a user, I want the website to load within three seconds even during high traffic periods.
PO	1) As a user, I want to be able to access the service from both a website and a mobile app, so that I can use it on multiple devices. 2) As a user, I want the app to work on both iOS and Android devices, so that I can use it regardless of my phone.
RL	1) As the IFA, we want the system to support at least 50,000 users simultaneously, so that it does not crash during peak times. 2) As an IT manager, I want the system to support data backups so that patient data is not lost in case of a system failure.
US	1) As a user, I want the app to save my preferences like language or format settings, so that I do not have to set them each time I use the service. 2) As a user, I want the site to have multilanguage support, so that I can navigate it in my preferred language.

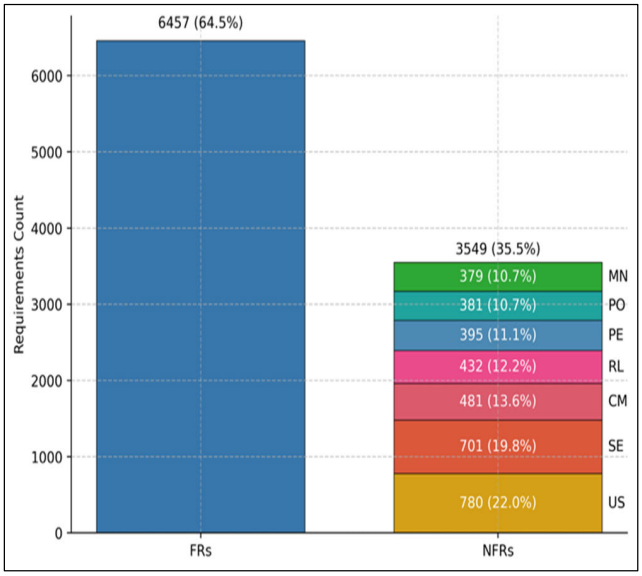

FIGURE 4. Dataset distribution.

TABLE 5. Selected models key characteristics.

Feature	BERT-Base-Uncased	DistilBERT-Base-Uncased	RoBERTa-Base	XLNet-Base-Cased
Provider	Google	Hugging Face	Facebook	Google/CMU
Architecture	Bidirectional Transformer	Distilled BERT (lighter)	Optimized BERT	Permutation Transformer
Pre-training Objective	MLM + NSP	MLM (no NSP, distillation)	MLM (no NSP, dynamic masking)	Permutation Language Model (PLM)
Parameters	110M	66M	125M	110M
Layers	12	6	12	12
Attention Heads	12	12	12	12
Vocabulary Size	30,522	30,522	50,265	32,000
Training Corpus	BooksCorpus, Wikipedia	Same as BERT	Common Crawl, OpenWebText	BooksCorpus, Wikipedia, Giga5, ClueWeb, Common Crawl

well-established pre-trained language models that serve as benchmarks in the field. We fine-tuned each model on our annotated NFR dataset to assess its effectiveness, adaptability to domain-specific syntax and terminology, and computational feasibility for automating our RE classification task. Table 5 provides a brief overview of the selected models.

1) BERT (BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS)

BERT [57] is a deep learning model that relies on the transformer encoder architecture which has been introduced by Vaswani et al. [58] Unlike previous sequential models, BERT uses self-attention to capture bidirectional contexts and understand words in relation to the surrounding tokens. It is trained using masked language modeling (MLM), where a portion of the input is randomly masked, and the model learns to predict missing words along with next-sentence prediction (NSP) to understand sentence relationships.

BERT was pre-trained on BooksCorpus and English Wikipedia, making it highly effective for syntactic and semantic understanding. This bidirectionality enables it to excel in many language tasks such as classification, sentiment analysis, named entity recognition, and question-answering. In our study, we fine-tuned BERT on a domain-specific dataset to classify NFRs in user stories, which often involve implicit, context-dependent language.

To compare the performance and efficiency of BERT, we also evaluated other variants of BERT.

2) DistilBERT (A DISTILLED VERSION OF BERT)

DistilBERT [59] is a lighter variant of BERT designed for computational efficiency, and it maintains ∼97% of BERT's performance of BERT on GLUE tasks using knowledge distillation. In addition, DistilBERT achieved a 40% lower computational cost by reducing the number of transformer layers from 12 to 6 and removing the NSP. This makes it well suited for resource-constrained environments.

3) RoBERTa (A ROBUSTLY OPTIMIZED BERT PRETRAINING APPROACH)

RoBERTa [60] is another BERT variant that aims to improve the BERT model. RoBERTa model removes the NSP objective, trained on a significantly larger corpus, and employs dynamic masking to change the masked tokens across different training epochs. This enhances its robustness for understanding complex sentence structures, making it a strong candidate for our classification task.

4) XLNet (GENERALIZED AUTOREGRESSIVE PRETRAINING FOR LANGUAGE UNDERSTANDING)

XLNet [61] employs permutation language modeling (PLM) instead of MLM, which dynamically permutes word order during training. Unlike BERT, XLNet integrates the prediction of the next tokens and bidirectional context modeling to enhance its contextual representations.

E. TOKENIZATION AND INPUT FORMATTING

Tokenization is a fundamental pre-processing step in modern NLP. Each pretrained model in this study utilizes its respective tokenizer to convert the dataset samples into token units suitable for model processing and is compatible with the model architecture. Before training a model on a dataset, the dataset must be pre-processed to the expected model input format. For this, tokenizers split the text into tokens that are converted into numerical representations known as embeddings and tensors, which become the model inputs. Because sentences do not always have the same lengths and the model input needs them to be in a uniform shape compatible with the model architecture, padding is used to add a special padding token [PAD] to shorter sequences so it can then have the right shape. By contrast, long sequences are truncated by the tokenizer to shorter lengths that the model can handle. The tokenizer uses a truncation parameter to truncate a sequence to the maximum length the model accepts. For a classification task, a classification token [CLS] is added at the beginning of every input sequence and used as a summary representation for classification tasks.

The tokenized sequences were transformed into numerical embeddings. The embeddings of BERT are categorized into the following three key components:

1. Token Embeddings: Capture the lexical and contextual meaning of each token.
2. Segment Embeddings: Differentiate between distinct input sequences when applicable (e.g., sentence-pair tasks). In single-sequence classification tasks (as in our case), the segment embeddings are typically set to the same value across all tokens.
3. Position embeddings: Encodes word order within the sequence, ensuring that the model maintains positional information despite using self-attention mechanisms.

These embedded representations are then summed to form the final input representation processed by multiple transformer layers that use self-attention mechanisms to extract contextual features. Figure 5 illustrates this BERT input representation, where the input embeddings result from the summation of the token embeddings, position embeddings, and, when applicable, segment embeddings.

Once the input embeddings are computed, they are processed through transformer layers, where self-attention mechanisms capture the long-range dependencies and contextual relationships. The final contextualized representation, particularly the [CLS] token, is passed through a classification head consisting of a fully connected layer, followed by a softmax activation function that produces a probability distribution over the eight requirement categories. Figure 6 illustrates an overview of how BERT processes input tokens for single-sentence classification tasks, where the final representation of the [CLS] token is used for classification.

The dataset was split into training and evaluation subsets to assess the performance of the models in the classification

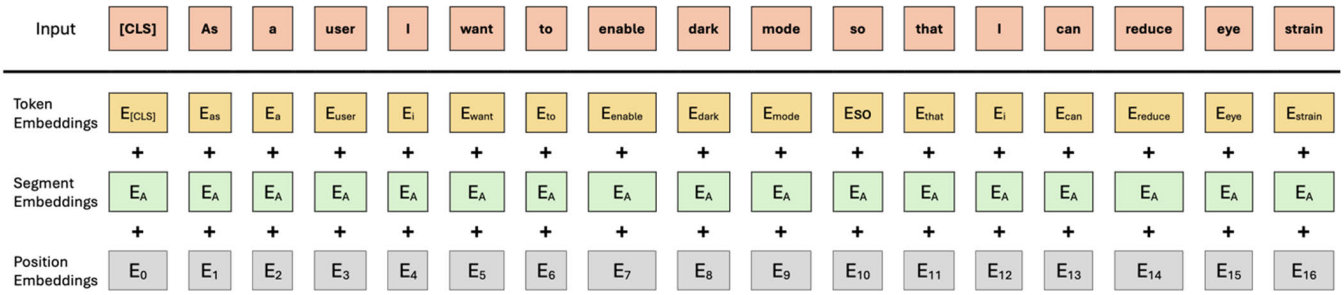

FIGURE 5. Bert input representation.

task. While this section describes BERT's tokenization and input representation, the other models used in this study—DistilBERT, RoBERTa, and XLNet—follow a similar process with slight variations. These differences primarily stem from their unique pre-training objectives and tokenizer implementations. For instance, RoBERTa removes the Next Sentence Prediction (NSP) task, leading to modifications in segment embeddings, whereas XLNet uses a permutation-based training approach that affects the token dependencies.

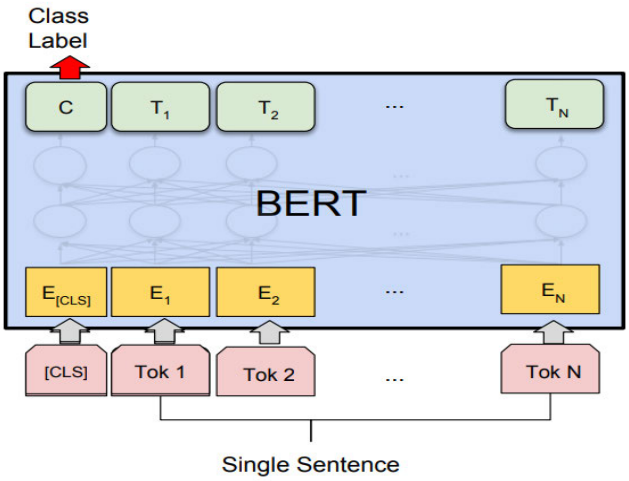

FIGURE 6. BERT Classification Process [57].

F. EXPERIMENTS SETUP

The experiments and implementation in this study were conducted in Google Colab's cloud-based computational environment. We utilized NVIDIA A100 and T4 GPUs through our Colab Pro subscription, alternating between them based on the available allocated resources, as some restrictions on computing units and time are applied to platform usage. This setup facilitates scalable access to high-performance hardware, ensuring the efficient execution of large-scale transformer-based models, thereby accelerating both training and evaluation.

The hugging-face transformers library was used to download tokenizers, perform tokenization, and initialize the models. Training and evaluation were conducted using Google Colab and PyTorch framework. We also employed Weights and Biases (WandB) [62] for the experimental tracking and hyperparameter logging. The dataset underwent preprocessing steps to standardize model inputs across training runs, including tokenization, padding, truncation, and label encoding, to ensure the optimization of transformer-based architectures.

To assess the predictive performance of the fine-tuned models, we used standard classification metrics including accuracy, precision, recall, and F1-score. The hardware and software configurations used in our experiments are summarized in Table 6.

TABLE 6. Hardwar aND software setup.

Component	Specification
Computational Platform:	Google Colab (Pro)
GPUs:	NVIDIA A100, NVIDIA T4
Storage:	Cloud-based
OS:	Cloud-based (Google Colab)
Programming Language:	Python 3.10
Deep Learning Framework:	PyTorch
NLP Library, Tokenizers, and Pretrained models:	Hugging Face Transformers[1]
Hyperparameter Optimization and Experiments Tracking:	Weights and Biases (WandB)[2]
Data Processing:	Pandas, NumPy, scikit-learn
Visualization:	Matplotlib, Seaborn

G. TRAINING PROCESS

In this section, we describe the training process of the selected models. The training started with data preprocessing, text cleaning, and splitting the dataset into a training set (80%) and a validation set (20%). The BERT [57], DistilBERT [59], RoBERTa [60], and XLNet [61] models were used with their respective tokenizers to tokenize texts and generate contextual embeddings to convert the input text into numerical representations. Next, tokenized sequences were passed through the transformer layers of each model. At this stage, the model's built-in multihead self-attention mechanism captured the semantic relationships between words. The final encoded representations from the transformer encoder are then fed into a fully connected linear layer, which maps them into the classification space.

The softmax activation function was used to calculate the final class probabilities by distributing the probabilities across the target classes, with the highest probability representing the predicted label. Because we have an imbalanced dataset, Focal Loss was leveraged instead of standard Cross-Entropy Loss. The focal loss helps reduce the dominance of easy-to-classify samples and assigns a higher weight to more complicated examples, thereby enhancing the model's generalization capability. The training hyperparameters were optimized using Bayesian Optimization on Weights and Biases (WandB) to determine the optimal learning rate, batch size, weight decay, and dropout rate. We used early stopping to stop the training if the evaluation loss did not improve for three consecutive epochs. Finally, the best-performing model for each approach was selected based on the lowest evaluation loss and the highest F1 score on the validation set. The subsequent sections provide more details on hyperparameter optimization and focal loss adjusting, followed by the evaluation process.

H. HYPERPARAMETERS OPTIMIZATION

Designing a well-performing ML model that meets a desired metric (e.g., high accuracy and minimal evaluation loss) is

[5]https://huggingface.co/docs/transformers/installation
[6]https://wandb.ai/site/experiment-tracking/

usually an iterative process that requires multiple experiments and evaluations. Tuning the hyperparameters in pre-trained models is crucial, as they can significantly impact the performance of the model. While pre-trained models have default baseline hyperparameter values, fine-tuning these parameters can lead to substantial improvements in performance.

There are several approaches for hyperparameter tuning. For instance, random hyperparameter search involves manual experimentation with different values. However, this method is time- and resource-intensive because the model must be retrained for each new hyperparameter configuration. Another common approach is a grid search, which systematically explores a predefined range of values. Similar to random search, grid search requires many experiments with different hyperparameter combinations, which makes it computationally expensive.

On the other hand, Bayesian Optimization offers a more efficient approach by employing a probabilistic model to determine the most promising hyperparameters. It iteratively refines its search by evaluating the values of a surrogate function before testing it on the actual model. Additionally, an early termination hyperband algorithm was applied to eliminate underperforming configurations, prevent unnecessary computations, and reduce the search time.

We used the Bayesian Optimization [63] setup available through the WandB platform to determine the optimal hyperparameters for fine-tuning the models. The advantage of Bayesian Optimization is its ability to exclude unpromising values early, thereby reducing the number of required experiments while accelerating the search for the best settings. We used Weights & Biases (WandB), a framework that facilitates intelligent hyperparameter optimization to efficiently manage and automate the hyperparameter-tuning process. The tuning process was executed using WandB's Bayesian search algorithm to determine the optimal set of hyperparameters. Table 7 outlines the range of hyperparameter values explored to obtain the best settings.

We run several experiments for each model to obtain the best combinations of hyperparameters for each model. Figures 7 to 10 illustrate selected experiments to show the correlation between the hyperparameters and F1-score. A notable improvement in performance between 5% and 8% was observed, significantly improving the final model results. Therefore, allocating additional effort and resources to tune the hyperparameters is worthwhile. Improving these values can substantially enhance the model's performance, as reflected in the research results and practical applications.

TABLE 7. Hyperparameters ranges and selected values.

Hyperparameter	Range	Model	Selected optimal value
Learning Rate	1e-5 – 5e-5	▪ BERT:	3e-05
		▪ DistilBERT:	1.23e-5
		▪ RoBERTa	4.3e-5
		▪ XLNet:	4.5e-5
Batch Size	8, 16, 32	▪ BERT:	32
		▪ DistilBERT:	8
		▪ RoBERTa	32
		▪ XLNet:	8
Number of Epochs	Early Stopping	▪ BERT:	10
		▪ DistilBERT:	10
		▪ RoBERTa	8
		▪ XLNet:	7
Weight Decay	0.01 – 0.05	▪ BERT:	0.02
		▪ DistilBERT:	0.03
		▪ RoBERTa	0.04
		▪ XLNet:	0.04
Dropout	0.05 – 0.5	▪ BERT:	0.06
		▪ DistilBERT:	0.1
		▪ RoBERTa	0.06
		▪ XLNet:	0.1

I. FOCAL LOSS

Imbalanced datasets are a significant challenge in general ML and fine-tuning pre-trained language models as well. Although large pre-trained language models are powerful in dealing with many classification problems, training on imbalanced datasets can lead to untrusted performance, as the models tend to be biased towards the dominant classes in the dataset. Consequently, the model's ability to generalize across different classes tends to be weakened.

The nature of the agile requirements in user stories is often functionally driven, which reflects the essence of user stories. Therefore, NFRs appear less in these specifications, which is common in Agile Requirements Engineering. Agile approaches rely heavily on the implicit knowledge of development teams when dealing with NFRs that are not explicitly documented. This was observed during the dataset labeling process and our labeling discussions, as Gilson et al. [45] pointed out in their work. However, some NFRs can be deduced from functional requirements analysis.

In this study, we used Focal Loss [64] to address the issue of class imbalance. Unlike traditional cross-entropy loss, Focal Loss dynamically reduces the influence of well-classified examples while placing greater emphasis on harder-to-classify samples. This approach helps minimize bias towards the dominant classes without requiring explicit resampling techniques. By down-weighting the loss assigned to well-classified examples, Focal Loss reshapes the standard cross-entropy loss and focuses training on challenging samples. This prevents easy negatives from overwhelming the model during training.

The focal loss approach determines two fundamental values: alpha (α) and gamma (γ), set to 0.25 and 2.0, respectively. These values are used to modify the Cross-Entropy Loss Function to reduce the influence of easy samples and enhance the attention to difficult samples, which helps improve the performance of the model on the underrepresented classes. Equation (1) defines the focal loss function as follows:

$$FL\left(p_t\right) = -\alpha_t \left(1 - p_t\right)^{\gamma} \log\left(p_t\right) \tag{1}$$

where:

- p_t is the model's estimated probability for the correct class.
- α_t is the weighting factor for the class imbalance used to control the importance of underrepresented classes.
- γ is the focusing parameter that reduces the influence of easy examples.
- $(1-p_t)^\gamma$ The scaling factor minimizes the influence of easy samples and downweighs well-classified examples to focus on hard examples.

When the sample is easily classified, the probability of correct classification is close to 1 ($p_t \approx 1$), and its influence is reduced $(1-p_t)^\gamma$. When the sample is difficult to classify ($small\ p_t$), its influence increases, prompting the model to focus more on it. This technique significantly improved the prevention of overfitting during training, resulting in a more balanced performance across all classes, especially underrepresented classes.

V. RESULTS

The softmax function was used to evaluate the performance of the studied models in our classification task. It calculates the probability for each class in a multiclass classification model. It was used to generate probability distributions over the eight classes in our datasets. Given an input agile user story, the model produces a vector of logits, which are then transformed into probabilities using the softmax function. For an input user story represented as a feature vector x and the corresponding weight vectors w, the softmax function (2) is defined as:

$$P(y=j\,|\,x) = \frac{e^{x^T w_j}}{\sum_{k=1}^{K} e^{x^T w_k}} \tag{2}$$

where:

- x is the input feature representation of the user story.
- w_j and w_k are the weight vectors for classes j and k, respectively.
- K is the total number of classes.

The softmax function ensures that the output probabilities sum to 1, allowing the model to assign a confidence score to each NFR category. The category with the highest probability is selected as the predicted label for the user story.

Because Focal Loss was used during training, it ensured that underrepresented NFR categories received higher weight, preventing the model from being biased toward the dominant functional requirements. Consequently, during the evaluation, the Softmax function helped interpret the model's learned representation, ensuring the predictions remained well-regulated across all categories. This combination of Focal Loss for training and softmax for evaluation significantly improves the model's ability to classify rare NFR categories, leading to a more balanced and robust classification performance across all eight labels.

We evaluated the performance of the investigated models using standard evaluation metrics for pre-trained models,

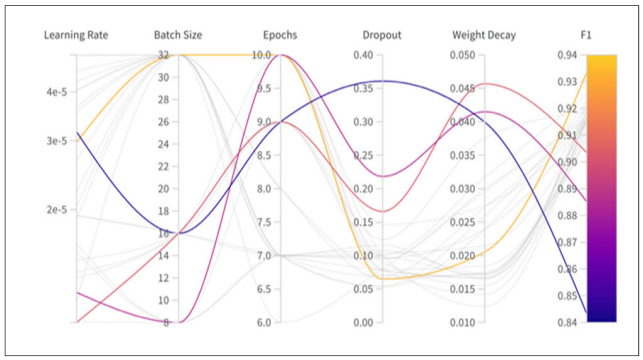

FIGURE 7. Hyperparameter combinations and correlation with respect to F1 (BERT Model).

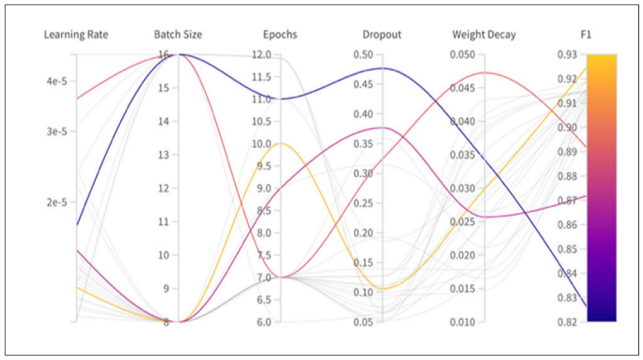

FIGURE 8. Hyperparameter combinations and correlation with respect to F1 (distilBERT Model).

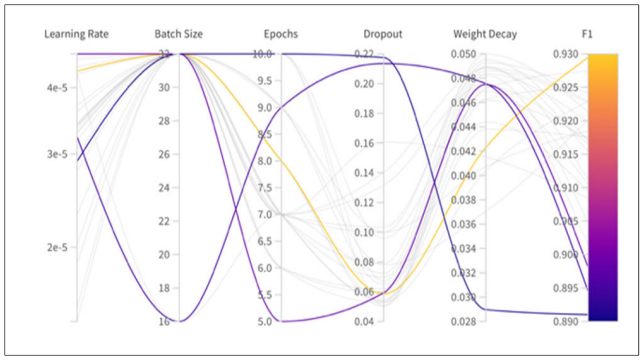

FIGURE 9. Hyperparameter combinations and correlation with respect to F1 (RoBERTa Model).

namely, Accuracy, Precision, Recall, and weighted $F1$-score. The accuracy measures the ratio of correct predictions to the total number of predictions. Precision measures the accuracy of positive predictions. The precision value indicates the model's confidence in its decisions when classifying NFRs. Recall measures the ability of a model to correctly identify all positive samples. A high recall value indicates that the model can successfully detect most NFRs, even if they are underrepresented in the dataset. The F1 Score combines the precision and recall into a single value using the harmonic mean to reliably evaluate the model, especially when dealing with unbalanced data. Finally, the weighted $F1$-score in

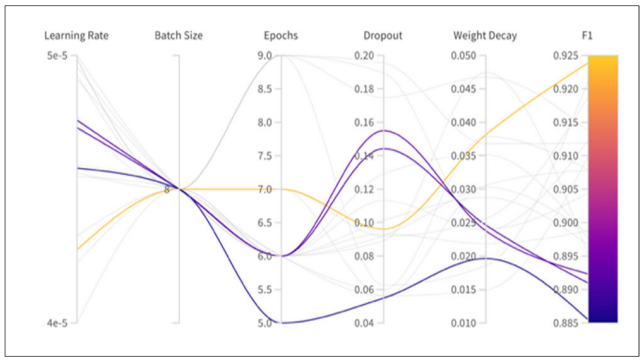

FIGURE 10. Hyperparameter combinations and correlation with respect to F1 (XLNet model).

Equation (7) was computed as the support-weighted average of the per-class $F1$-scores defined in Equation (6), with each score weighted by its class prevalence (support). The evaluation metrics (3)-(7) were calculated as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{4}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{5}$$

$$F1\text{Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{6}$$

$$\text{Weighted } F1 = \sum_{i=1}^{N} \left(\frac{\text{Support}_i}{\text{Total}} \times F1_i \right) \tag{7}$$

where:

- TP (True Positives): The number of correctly classified positive samples.
- TN (True Negatives): The number of correctly classified negative samples.
- FP (False Positives): The number of negative samples incorrectly classified as positive.
- FN (False Negatives): The number of positive samples incorrectly classified as negative.
- $F1_i$: F1-score of class i
- Support_i: number of true samples in class i
- Total: total number of samples across all classes

As the dataset is imbalanced and the task involves eight categories, we report weighted averages for precision, recall, and F1-score. Weighted averaging ensures that each class contributes proportionally to its frequency, reflecting model performance more accurately under class imbalance. Table 8 reports the weighted average performance metrics (Accuracy, Precision, Recall, and weighted $F1$-Score) for each of the four fine-tuned pretrained models, reflecting their ability to classify NFRs in an imbalanced eight-class setting. Our results indicated the excellent performance of these models in identifying NFRs in user stories. These findings suggest that pre-trained models could provide a viable solution to the challenge of neglecting NFRs, thus addressing the user stories' inability to effectively represent them.

TABLE 8. Weighted average performance metrics across classes.

MODEL	Accuracy	Precision	Recall	Weighted $F1$-Score
MODEL	**Accuracy**	**Precision**	**Recall**	**F1**
BERT	**0.9341**	**0.9337**	**0.9341**	**0.9335**
DistilBERT	0.9246	0.9257	0.9246	0.9248
RoBERTa	0.9291	0.9307	0.9291	0.9295
XLNet	0.9149	0.9152	0.9149	0.9147

When benchmarking against previous relevant research, as shown in Table 9, our comparison demonstrates that our results outperform those of prior studies. This superior performance highlights the promising potential of pre-trained models in the field of agile RE, paving the way for more advanced applications.

TABLE 9. Comparison of proposed approaches with SOTA approaches.

Study	Dataset	Precision	Recall	$F1$
Our study (BERT Model)	**Agile**	**0.9337**	**0.9341**	**0.9335**
Gilson et al. [45]	Agile	0.74	0.41	0.53
Galster et al. [46]	Agile	0.65	0.67	0.66
Ahmad et al [43]	Agile	0.79	0.92	0.85
Li and Nong [33]	N/A	0.9100	0.9265	0.9147
Khan et al [65]	N/A	0.9148	0.9148	0.9148

Figures 11 to 14 present both the normalized and absolute confusion matrices for the evaluated pre-trained models. The results demonstrate strong performance in predicting FRs and various NFRs, with BERT showing a slight advantage over the other models. To facilitate clearer interpretation, we report the confusion matrices using normalized values (percentages), which help highlight classification performance across all categories, particularly under class imbalance. The absolute counts are also displayed above the normalized percentages within each cell.

VI. DISCUSSION AND IMPLICATIONS

Our results demonstrate that transfer learning through pre-trained language transformer-based models can facilitate and enhance the identification and classification of explicit and implicit NFRs in software requirements specified using user stories. The performance of the pre-trained BERT model showed excellent F1 accuracy. The other investigated models exhibited acceptable performances. Our approach provides a more scalable and generalizable solution than previous studies that used rule-based classifiers and traditional machine learning. One of the critical challenges encountered was the imbalance in the representation of NFRs in agile datasets. While applying Focal Loss to mitigate this challenge in our study, some underrepresented NFR categories, such

FIGURE 11. BERT model.

FIGURE 12. DISTILBERT model.

FIGURE 13. RoBERTa model.

FIGURE 14. XLNET model.

as portability and maintainability, still showed marginally lower recall scores. Therefore, additional data augmentation techniques, such as the synthetic minority oversampling technique SMOTE [66], may be helpful in further enhancing model performance.

Our findings also highlight the correlation between model performance and dataset quality. In addition, the variation in how NFRs are expressed across projects may affect the generalizability of the model; however, this can be addressed by considering historical data for different projects, systems, and domains. It is also necessary to allocate some investment in hyperparameter optimization when working with pre-trained models, as hyperparameters significantly improve the model performance.

Our results have several important implications for ASD and agile RE. One implication is that automating NFR identification and classification could provide important support for reducing the manual effort required to identify NFRs in agile development environments, leading to a more structured, consistent, and rapid approach to agile NFR documentation. In addition, earlier identification and consideration of NFRs allows teams to include essential NFRs and quality attributes in their development lifecycle, which might mitigate and prevent later costly rework.

The proposed approach can be feasibly embedded into agile requirement management tools to provide real-time NFR identification and thus support informed decision-making for both product owners and developers with the acknowledgment that adoption of such models by agile teams may face some challenges with regard to the interpretation of the model outputs and integration into agile RE workflows. Therefore, further research is needed to explore how such a process and its trustworthiness improve automated NFR identification and classification. Additional

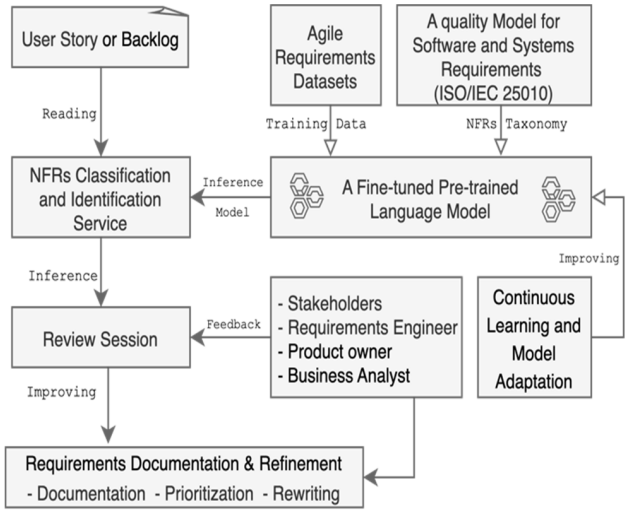

FIGURE 15. Envisioned deployment pipeline in practice.

work is also needed on how such an approach can facilitate requirements prioritization and documentation as an acceptance criteria format, for instance, which is one of our considered future works.

Figure 15 illustrates the envisioned practical pipeline for deploying a fine-tuned pre-trained language model for NFR identification and classification in Agile RE. The process begins with user stories or backlog items, which are analyzed by a classification and identification service powered by a pre-trained model adapted to the NFR taxonomy defined in ISO/IEC 25010. The model is trained on annotated agile datasets and refined through iterative feedback loops involving stakeholders such as requirements engineers and product owners. Post-classification, a review session allows for validation and human oversight. The results feed into formal requirements documentation, prioritization, and rewriting. The system supports continuous learning and model adaptation through updated training data and refined requirements artifacts, ensuring long-term robustness and domain relevance. In future work, we aim to implement this envisioned pipeline within real agile development teams to study its practical integration, usability, and impact on NFR identification and documentation.

VII. CONCLUSION

This study used pre-trained language models to identify and classify NFRs in agile-based requirements. We prepared and labeled an agile requirements-based dataset for this study. We then finetuned several language models to the research task, taking advantage of the transfer learning capabilities provided by the pre-trained language models. We also conducted several experiments to optimize the models training hyperparameters to enhance their performance. To address the problem of dataset imbalance, we used Focal Loss to ensure that the dominant classes did not contribute to a bias by models towards them, ensuring consideration of the less represented classes. Our results showed the superiority of the pre-trained models in identifying and classifying NFRs in user stories over previous studies that relied on roll-based systems or traditional machine-learning models. These results open a variety of research directions and applications for integrating pre-trained language models into the workflow for Agile RE to address the problem of neglecting NFRs in Agile RE, which reduces the cost of changing requirements later, ensuring quality aspects early and preparing the environment for informed decisions about requirements. The results of this approach can also be used for other applications related to prioritizing requirements and formulating acceptance criteria for NFRs in user stories, which we intend to investigate in future studies.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support of the Department of Information Systems, King Saud University (KSU), and King Khalid University (KKU). They also sincerely thank Abdelrazek Elshemy from ELM Company for his valuable assistance and insightful feedback during the dataset labeling and review process, which significantly contributed to the quality of this research.

REFERENCES

[1] Digital. AI. (2022). *16th Annual State of Agile Survey Report*. [Online]. Available: https://info.digital.ai/rs/981-LQX-968/images/AR-SA-2022-16th-Annual-State-Of-Agile-Report.pdf

[2] R. Hoda, N. Salleh, and J. Grundy, "The rise and evolution of agile software development," *IEEE Softw.*, vol. 35, no. 5, pp. 58–63, Sep. 2018, doi: 10.1109/MS.2018.290111318.

[3] S. Stavru, "A critical examination of recent industrial surveys on agile method usage," *J. Syst. Softw.*, vol. 94, pp. 87–97, Aug. 2014, doi: 10.1016/j.jss.2014.03.041.

[4] V. T. Heikkilä, D. Damian, C. Lassenius, and M. Paasivaara, "A mapping study on requirements engineering in agile software development," in *Proc. 41st Euromicro Conf. Softw. Eng. Adv. Appl.*, Aug. 2015, pp. 199–207, doi: 10.1109/SEAA.2015.70.

[5] I. Inayat, S. S. Salim, S. Marczak, M. Daneva, and S. Shamshirband, "A systematic literature review on agile requirements engineering practices and challenges," *Comput. Hum. Behav.*, vol. 51, pp. 915–929, Oct. 2015, doi: 10.1016/j.chb.2014.10.046.

[6] B. Ramesh, L. Cao, and R. Baskerville, "Agile requirements engineering practices and challenges: An empirical study," *Inf. Syst. J.*, vol. 20, no. 5, pp. 449–480, Sep. 2010, doi: 10.1111/j.1365-2575.2007.00259.x.

[7] E. M. Schön, D. Winter, M. J. Escalona, and J. Thomaschewski, "Key challenges in agile requirements engineering," in *Proc. 18th Int. Conf. Agile Processes Softw. Eng. Extreme Program.*, vol. 283, 2017, pp. 37–51, doi: 10.1007/978-3-319-57633-6_3.

[8] W. Alsaqaf, M. Daneva, and R. Wieringa, "Quality requirements in large-scale distributed agile projects—A systematic literature review," in *Proc. Requirements Eng., Found. Softw. Quality (REFSQ)*, P. Grünbacher and A. Perini, Eds., Jan. 2017, pp. 219–234, doi: 10.1007/978-3-319-54045-0_17.

[9] J. Medeiros, D. C. P. Alves, A. Vasconcelos, C. Silva, and E. G. Wanderley, "Requirements engineering in agile projects: A systematic mapping based in evidences of industry," in *Proc. 18th Ibero-Amer. Conf. Softw. Eng.*, Jan. 2015, pp. 460–473.

[10] X. Wang, L. Zhao, Y. Wang, and J. Sun, "The role of requirements engineering practices in agile development: An empirical study," in *Requirements Engineering*, D. Zowghi and Z. Jin, Eds., Berlin, Germany: Springer, 2014, pp. 195–209, doi: 10.1007/978-3-662-43610-3_15.

[11] M. Kassab, "The changing landscape of requirements engineering practices over the past decade," in *Proc. IEEE 5th Int. Workshop Empirical Requirements Eng. (EmpiRE)*, Aug. 2015, pp. 1–8, doi: 10.1109/EMPIRE.2015.7431299.

[12] E.-M. Schön, J. Thomaschewski, and M. J. Escalona, "Agile requirements engineering: A systematic literature review," *Comput. Standards Interfaces*, vol. 49, pp. 79–91, Jan. 2017, doi: 10.1016/j.csi.2016.08.011.

[13] J. Nawrocki, M. Ochodek, J. Jurkiewicz, S. Kopczyńska, and B. Alchimowicz, "Agile requirements engineering: A research perspective," in *Proc. Int. Conf. Current Trends Theory Practice Inform.*, V. Geffert, B. Preneel, B. Rovan, J. Štuller, and A. M. Tjoa, Eds., 2014, pp. 40–51, doi: 10.1007/978-3-319-04298-5_5.

[14] M. Kassab, C. Neill, and P. Laplante, "State of practice in requirements engineering: Contemporary data," *Innov. Syst. Softw. Eng.*, vol. 10, no. 4, pp. 235–241, Dec. 2014, doi: 10.1007/s11334-014-0232-4.

[15] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, "Forging high-quality user stories: Towards a discipline for agile requirements," presented at the IEEE 23rd Int. Requirements Eng. Conf. (RE), Aug. 2015, doi: 10.1109/RE.2015.7320415.

[16] F. Dalpiaz, A. Ferrari, X. Franch, and C. Palomares, "Natural language processing for requirements engineering: The best is yet to come," *IEEE Softw.*, vol. 35, no. 5, pp. 115–119, Sep. 2018, doi: 10.1109/MS.2018.3571242.

[17] F. Dalpiaz, "Requirements data sets (user stories)," Mendeley Data, London, U.K., Tech. Rep. 7zbk8zsd8y.1, 2018, doi: 10.17632/7zbk8zsd8y.1.

[18] M. Beedle. *Manifesto for Agile Software Development*. Accessed: Feb. 2, 2022. [Online]. Available: https://www.agilemanifesto.org

[19] F. Paetsch, A. Eberlein, and F. Maurer, "Requirements engineering and agile software development," in *Proc. 12th IEEE Int. Workshops Enabling Technol., Infrastruct. Collaborative Enterprises*, Jun. 2003, pp. 308–313, doi: 10.1109/ENABL.2003.1231428.

[20] L. Cao and B. Ramesh, "Agile requirements engineering practices: An empirical study," *IEEE Softw.*, vol. 25, no. 1, pp. 60–67, Jan. 2008, doi: 10.1109/MS.2008.1.

[21] G. Lucassen, F. Dalpiaz, J. M. E. M. V. D. Werf, and S. Brinkkemper, "The use and effectiveness of user stories in practice," in *Proc. 22nd Int. Work. Conf. Requirements Eng., Found. Softw. Quality*, Berlin, Germany: Springer, Jan. 2016, pp. 205–222, doi: 10.1007/978-3-319-30282-9_14.

[22] M. Cohn, *User Stories Applied: For Agile Software Development*. Reading, MA, USA: Addison-Wesley, 2004.

[23] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, "Improving agile requirements: The quality user story framework and tool," *Requirements Eng.*, vol. 21, no. 3, pp. 383–403, Sep. 2016, doi: 10.1007/s00766-016-0250-x.

[24] G. Lucassen, M. Robeer, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, "Extracting conceptual models from user stories with visual narrator," *Requirements Eng.*, vol. 22, no. 3, pp. 339–358, Sep. 2017, doi: 10.1007/s00766-017-0270-1.

[25] I. Sommerville, *Requirements Engineering*, 10th ed., London, U.K.: Pearson, 2016, pp. 101–137.

[26] M. Glinz, "On non-functional requirements," in *Proc. 15th IEEE Int. Requirements Eng. Conf. (RE)*, Oct. 2007, pp. 21–26, doi: 10.1109/RE.2007.45.

[27] L. Chung and J. C. S. do Prado Leite, "On non-functional requirements in software engineering," in *Conceptual Modeling: Foundations and Applications*. Berlin, Germany: Springer, 2009, pp. 363–379, doi: 10.1007/978-3-642-02463-4_19.

[28] I. K. Raharjana, D. Siahaan, and C. Fatichah, "User stories and natural language processing: A systematic literature review," *IEEE Access*, vol. 9, pp. 53811–53826, 2021, doi: 10.1109/ACCESS.2021.3070606.

[29] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, Dec. 2024, doi: 10.1145/3695988.

[30] J. Cleland-Huang, R. Settimi, X. Zou, and P. Solc, "Automated classification of non-functional requirements," *Requirements Eng.*, vol. 12, no. 2, pp. 103–120, Apr. 2007, doi: 10.1007/s00766-007-0045-1.

[31] J. Slankas and L. Williams, "Automated extraction of non-functional requirements in available documentation," in *Proc. 1st Int. Workshop Natural Language Anal. Softw. Eng. (NaturaLiSE)*, May 2013, pp. 9–16, doi: 10.1109/NATURALISE.2013.6611715.

[32] N. Handa, A. Sharma, and A. Gupta, "Framework for prediction and classification of non functional requirements: A novel vision," *Cluster Comput.*, vol. 25, no. 2, pp. 1155–1173, Apr. 2022, doi: 10.1007/s10586-021-03484-0.

[33] B. Li and X. Nong, "Automatically classifying non-functional requirements using deep neural network," *Pattern Recognit.*, vol. 132, Dec. 2022, Art. no. 108948, doi: 10.1016/j.patcog.2022.108948.

[34] A. Silva, P. R. Pinheiro, A. B. Albuquerque, and J. Barroso, "A process for creating the elicitation guide of non-functional requirements," in *Proc. Int. Conf. Inf. Commun. Technol. Intell. Syst.*, Cham, Switzerland: Springer, Jan. 2016, pp. 293–302, doi: 10.1007/978-3-319-33622-0_27.

[35] S. Kopczynska and J. Nawrocki, "Using non-functional requirements templates for elicitation: A case study," in *Proc. IEEE 4th Int. Workshop Requirements Patterns (RePa)*, Aug. 2014, pp. 47–54, doi: 10.1109/REPA.2014.6894844.

[36] W. M. Farid, "The NORMAP methodology: Lightweight engineering of non-functional requirements for agile processes," in *Proc. 19th Asia–Pacific Softw. Eng. Conf.*, vol. 1, Dec. 2012, pp. 322–325, doi: 10.1109/APSEC.2012.23.

[37] D. Domah and F. J. Mitropoulos, "The NERV methodology: A lightweight process for addressing non-functional requirements in agile software development," in *Proc. SoutheastCon*, Apr. 2015, pp. 1–7, doi: 10.1109/SECON.2015.7133028.

[38] W. M. Farid and F. J. Mitropoulos, "NORPLAN: Non-functional requirements planning for agile processes," in *Proc. IEEE Southeastcon*, Apr. 2013, pp. 1–8, doi: 10.1109/SECON.2013.6567463.

[39] W. Behutiye, P. Rodríguez, and M. Oivo, "Quality requirement documentation guidelines for agile software development," *IEEE Access*, vol. 10, pp. 70154–70173, 2022, doi: 10.1109/ACCESS.2022.3187106.

[40] M. Younas, D. N. A. Jawawi, I. Ghani, and R. Kazmi, "Non-functional requirements elicitation guideline for agile methods," *J. Telecommun. Electron. Comput. Eng.*, vol. 9, nos. 3–4, pp. 137–142, Oct. 2017.

[41] M. Younas, D. N. A. Jawawi, M. A. Shah, A. Mustafa, M. Awais, M. K. Ishfaq, and K. Wakil, "Elicitation of nonfunctional requirements in agile development using cloud computing environment," *IEEE Access*, vol. 8, pp. 209153–209162, 2020, doi: 10.1109/ACCESS.2020.3014381.

[42] E. Sherif, W. Helmy, and G. H. Galal-Edeen, "Proposed framework to manage non-functional requirements in agile," *IEEE Access*, vol. 11, pp. 53995–54005, 2023, doi: 10.1109/ACCESS.2023.3281195.

[43] M. Ahmed, S. U. R. Khan, and K. A. Alam, "QAExtractor: A quality attributes extraction framework in agile-based software development," in *Proc. 1st Int. Workshop Intell. Softw. Autom.*, Singapore, S. Anwar and A. Rauf, Eds., Springer, Jan. 2021, pp. 15–28.

[44] M. Ahmed, S. U. R. Khan, and K. A. Alam, "An NLP-based quality attributes extraction and prioritization framework in agile-driven software development," *Automated Softw. Eng.*, vol. 30, no. 1, p. 7, Jan. 2023, doi: 10.1007/s10515-022-00371-9.

[45] F. Gilson, M. Galster, and F. Georis, "Extracting quality attributes from user stories for early architecture decision making," in *Proc. IEEE Int. Conf. Softw. Archit. Companion (ICSA-C)*, Mar. 2019, pp. 129–136, doi: 10.1109/ICSA-C.2019.00031.

[46] M. Galster, F. Gilson, and F. Georis, "What quality attributes can we find in product backlogs? A machine learning perspective," in *Proc. Eur. Conf. Softw. Archit. (ECSA)*, vol. 2019, Cham, Switzerland, T. Bures, L. Duchien, and P. Inverardi, Eds., Springer, pp. 88–96.

[47] H. Villamizar, M. Kalinowski, A. Garcia, and D. Mendez, "An efficient approach for reviewing security-related aspects in agile requirements specifications of Web applications," *Requirements Eng.*, vol. 25, no. 4, pp. 439–468, Dec. 2020, doi: 10.1007/s00766-020-00338-w.

[48] F. Casillo, V. Deufemia, and C. Gravino, "Detecting privacy requirements from user stories with NLP transfer learning models," *Inf. Softw. Technol.*, vol. 146, Jun. 2022, Art. no. 106853, doi: 10.1016/j.infsof.2022.106853.

[49] G. B. Herwanto, G. Quirchmayr, and A. M. Tjoa, "Leveraging NLP techniques for privacy requirements engineering in user stories," *IEEE Access*, vol. 12, pp. 22167–22189, 2024, doi: 10.1109/ACCESS.2024.3364533.

[50] P. K. Murukannaiah, N. Ajmeri, and M. P. Singh, "Acquiring creative requirements from the crowd: Understanding the influences of personality and creative potential in crowd RE," in *Proc. IEEE 24th Int. Requirements Eng. Conf. (RE)*, Sep. 2016, pp. 176–185, doi: 10.1109/RE.2016.68.

[51] S. G. Köse and F. B. Aydemir, "A user story dataset for library and restaurant management systems," Zenodo, Geneva, Switzerland, Tech. Rep., 2023, doi: 10.5281/ZENODO.7529130.

[52] A. Rohmann and B. Paech, "Dataset and application of algorithms of 'towards a more set of acceptance criteria,'" Zenodo, Geneva, Switzerland, Nov. 2022, doi: 10.5281/ZENODO.7358697.

[53] A. Prabu, L. A. Suruthi, B. Sabarish, and K. Arun, "User story-based automatic test case classification and prioritization using natural language processing-based deep learning," *Proc. IEEE Potentials*, vol. 43, no. 5, pp. 20–28, Sep. 2024, doi: 10.1109/MPOT.2023.3342366.

[54] F. Dalpiaz, P. Gieske, and A. Sturm, "On deriving conceptual models from user requirements: An empirical study," *Inf. Softw. Technol.*, vol. 131, Mar. 2021, Art. no. 106484, doi: 10.1016/j.infsof.2020.106484.

[55] *Systems and Software Engineering–Systems and Software Quality Requirements and Evaluation (SQuaRE)–System and Software Quality Models*, Standard ISO/IEC 25010:2011, 2011.

[56] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, Apr. 1960.

[57] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. for Comput. Linguistics, Human Language Technol.*, J. Burstein, C. Doran, and T. Solorio, Eds., Jun. 2019, pp. 4171–4186, doi: 10.18653/v1/n19-1423.

[58] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.

[59] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.

[60] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.

[61] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le, "XLNet: Generalized autoregressive pretraining for language understanding," in *Proc. Adv. Neural Inf. Process. Syst.*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., Jan. 2019, pp. 1–15.

[62] L. Biewald. (2020). *Experiment Tracking With Weights and Biases*. [Online]. Available: https://www.wandb.com/

[63] I. Dewancker, M. McCourt, and S. Clark, "Bayesian optimization for machine learning: A practical guidebook," 2016, *arXiv:1612.04858*.

[64] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2999–3007.

[65] M. A. Khan, M. S. Khan, I. Khan, S. Ahmad, and S. Huda, "Non functional requirements identification and classification using transfer learning model," *IEEE Access*, vol. 11, pp. 74997–75005, 2023, doi: 10.1109/ACCESS.2023.3295238.

[66] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.

ABDULRAHIM ALHAIZAEY received the Bachelor of Science degree in information systems from King Khalid University, Saudi Arabia, and the master's degree in computer science from George Washington University, USA. He is currently pursuing the Ph.D. degree in information systems with the College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia. He was a Teaching Assistant with the Information Systems Department, King Khalid University, from 2010 to 2012, and a Lecturer, from 2014 to 2020, before joining his Ph.D. research. His research interests include requirements engineering, systems analysis and design, and the application of artificial intelligence techniques to address the research challenges in these areas.

MAJED AL-MASHARI (Member, IEEE) is currently a Professor of information systems with King Saud University, with extensive expertise in information technology, software engineering, and business process management. He provides consultations in enterprise systems, ERP, CRM, e-commerce, e-Government, and IT strategic planning. He has published numerous scientific papers in highly reputable journals and international conferences. His research interests include business process management, enterprise resource planning (ERP) systems, requirements engineering, and intelligent information systems. He has received several international and regional scientific awards in recognition of his contribution to the field. He actively serves as a reviewer for leading scientific journals and research theses and has been involved in international scientific committees for worldwide IT conferences. He served as the Editor-in-Chief for the *Business Process Management Journal* and held editorial board positions in several renowned international journals. He has played a significant role in shaping the IT research landscape and has chaired multiple national and international IT-related conferences and forums. He is a frequent speaker in public lectures and industry events, contributing to advancing knowledge in business processes and IT management, digital transformation, software engineering, and information systems.